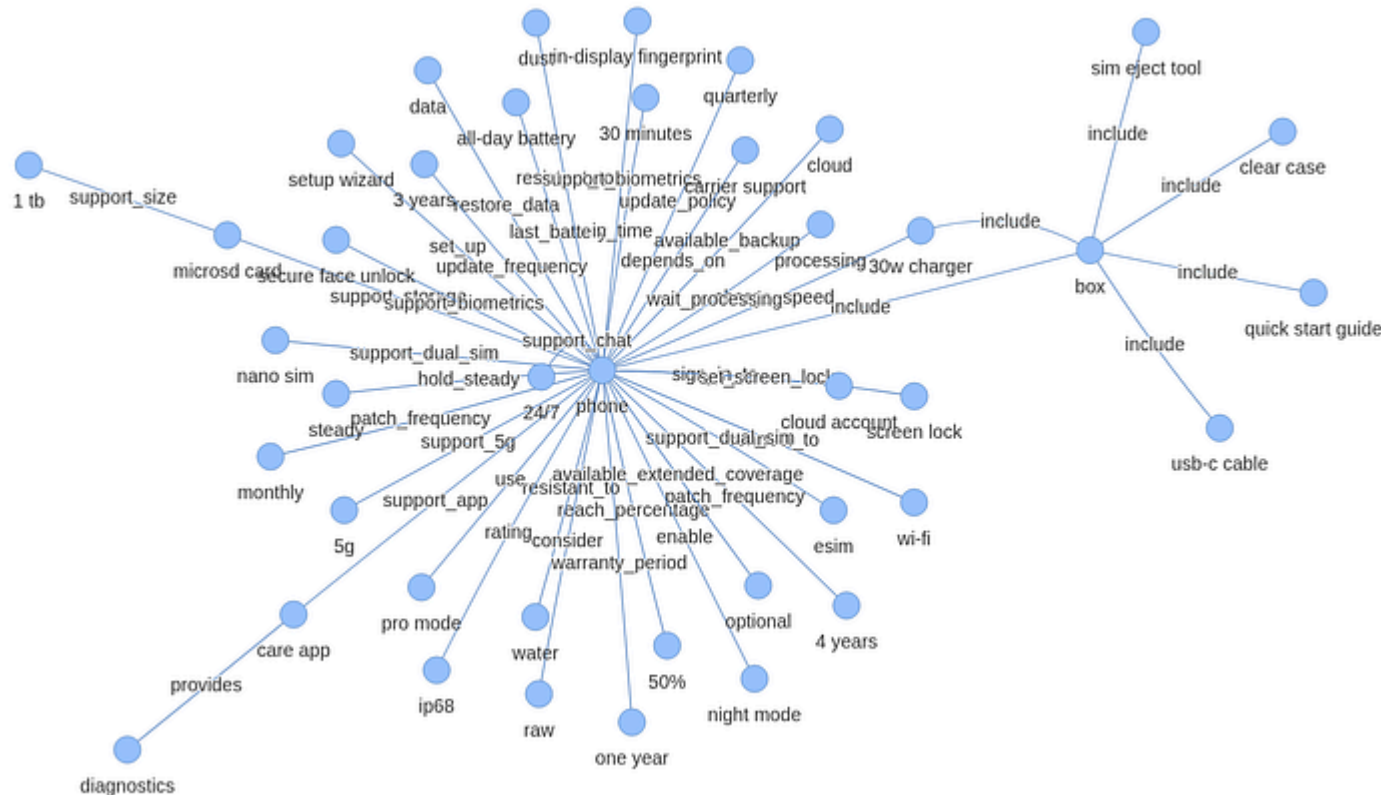


[< Go to the original](#)

How to Build a QA System using Knowledge Graph and LLM

Use LLMs to build a QA system on top of Knowledge Graphs



Follow



The Muse Junction a11y-light ~6 min read · September 6, 2025 (Updated: September 6, 2025) ·

Free: No

As promised in my earlier article on "[How to Build Knowledge Graphs using LLMs on Local Machine](#)", I'm back with a simple Question Answer (QA) System using a knowledge graph (built with the approach we discussed in the first part) and an LLM (in this case Gemma3-4b-it-qat, same one we used earlier). I chose to work with Gemma3-4b because it's small enough to run on consumer laptops, yet it is excellent at following instructions. If you need more information and context on KG, Gemma3 or how to run LLMs in local machine I would recommend you skim through my previous article.

In this blog, we will take a sample FAQ text about a hypothetical smartphone, generate knowledge graph for it using previous article's code (same repository), then build a system to answer questions related to the product, like this:

```
-----  
-----  
query: does the phone come with charger?  
Answer:Yes, the phone comes with a 30w charger.  
-----  
-----  
query: does the phone have cloud backup?  
Answer:Yes, the phone has cloud backup.  
-----
```

some questions and answers

In this article, we will learn:

1. What is a QA System
2. Understand our approach
3. Look at the code
4. Discuss limitations and further improvements

What is a QA System

Quoting Google:

A question answering (QA) system is a software application that takes a user's question in natural language and provides a direct, relevant answer by processing the question's intent and retrieving information from a knowledge source or generating a new response.

blog. We can run the main.py file of the repository to form a KG and save it to output directory:

Copy

```
python main.py --inputpath ./input/sample-faq.txt --outlabel faq
```

This command saves the networkx graph in a file "nx_graph.pkl" which we would later load for QA system building.

Our Approach

The core idea is to extract the entities/keywords from the given question, find all the nodes and edges related to them and then feed it to the LLM to find the answer based on information from KG and question. Let's formalise the solution below.

Given a question q and a knowledge graph (G) built on any corpus:

1. Use LLM to extract list of named entities (*entity_keywords*) and relations/predicates (*relation_keywords*) from q .

be source or target.

3. For each pair (u, v) from step 2, find all paths in G between u & v . This allows us to find all the relations/paths/knowledge between two entities. Introducing this step was a game-changer.
4. For each path found (source and target node), find the relation between them. E.g. (box, include, charger)
5. Add the formed "triple" to a list, *relations*.
6. We need to do something similar for the *relation_keywords*. For each relation r find all the edges which are connected by r . Form triples and add it to same list, *relations*.
7. Final step is to pass the triples and the question q , wrapped in a prompt, to the LLM to form a response based on given facts (triples) and query.

Code Implementation

First, we need to build the knowledge graph using main.py file:

Copy

```
python main.py --inputpath ./input/sample-faq.txt --outlabel faq
```

Copy

```
import pickle

G = pickle.load(open(graph_file, "rb"))
```

We need to define a function which can take our text input and system level prompt and give a response from LLM. Following reusable function does the same

Copy

```
def get_llm_response(text, system_prompt):
    response = ollama.chat(model=model, messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": text}
    ])
    resp_content = response['message']['content']
    return resp_content
```

We need to extract entities and relations from given query (step 1 from above). For this I've formed a basic system prompt as shown below.

```

system_prompt_key_words = "You are a helpful assistant, expert on English language. The returned keywords should be critical to answer the question. Categorize the keywords into entity and relation keywords. keywords must be in root form and lowercase. The response should be in following format, no additional text: {'entity': [list of entity keywords], 'relation': [list of relation keywords]}".

response = get_llm_response(query, system_prompt_key_words)
keyword_resp = json.loads(response)
entity_keywords = keyword_resp.get('entity', [])
relation_keywords = keyword_resp.get('relation', [])

```

Suppose, *entity_keywords* is [box, charger, phone], we need to find all possible source-target pairs so that we can query graph for all:

Copy

```
pairs = list(combinations(entities, 2))
```

For each entity pair, we need to query the graph to find all the nodes and edges

Copy

```
paths = list(nx.all_simple_paths(G, source=u, target=target_nodes))
```

Copy

```
def search_kg2(G, query):
    response = get_llm_response(query, system_prompt_key_words)
    keyword_resp = json.loads(response)
    entity_keywords = keyword_resp.get('entity', [])
    relation_keywords = keyword_resp.get('relation', [])
    entities = [part.strip() for part in entity_keywords]
    pairs = list(combinations(entities, 2))
    relations = []
    for u, v in pairs:
        target_nodes = get_nodes(G, v)
        paths = list(nx.all_simple_paths(G, source=u, target=target_nodes))
        for path in paths:
            for i in range(len(path)-1):
                for key in G[path[i]][path[i+1]]:
                    rel = G[path[i]][path[i+1]][key]['relation']
                    relations.append((path[i], rel, path[i+1]))

    for rel_keyword in relation_keywords:
        relations.extend([(u, rel, v) for u, v, rel in G.edges.data("relation")
                           if rel == rel_keyword])

    return relations
```

Once we have all the edges represented as triples (entity->relation->entity) from above function, we pass these to LLM embedded in a instructive prompt:


```
context = f'''
    You are given facts from a knowledge graph:

    {triples}

    Answer the user query based ONLY on these facts.
    Answer in full sentence.
    Query: {query}
    '''

response = ollama.chat(model="gemma3:4b-it-qat",
                       messages=[{"role": "user", "content": context}])
print(f'query: {query}\nAnswer:{response["message"] ["content"]}')
```

This returns answers like these:

```
-----
query: does the phone come with charger?
Answer:Yes, the phone comes with a 30w charger.
-----
-----
query: does the phone have cloud backup?
Answer:Yes, the phone has cloud backup.
-----
-----
query: does the phone support 5G?
Answer:Yes, the phone supports 5G.
-----
-----
query: is there a headphone jack?
Answer:The provided facts do not contain information about headphone jacks.
-----
-----
query: does the phone support wireless charging?
Answer:I am sorry, I cannot answer the question about whether the phone supports wireless charging because no facts were provided i
-----
-----
query: what is the screen size?
Answer:I am sorry, but I cannot answer the query "what is the screen size?" because no facts about screen size were provided in the
-----
```

As you can see LLM gracefully refuses to answer the questions for which it doesn't have any data/facts.

The full code for this blog can be found in [qa-from-kg.ipynb](#) file.

Limitations

As we have seen above, with less effort we were able to create a basic QA system because LLM was able to do a lot of heavy lifting, like text pre-processing, extraction etc. But it's not perfect yet. During my initial evaluations I found at least a few limitations.

system failed to find any edges. So our core system prompt to form knowledge graph needs more work.

There were a few questions like these which couldn't be answered by the system until they were rephrased a little. But, I found that all of these boiled down to the way KG was formed or keyword extracted from query. Both of these are fixable by improving prompts. For instance, one of the edges on the graph I was working on was:

```
phone → support_dual_sim → nano sim
```

which is clearly not ideal. But all of these could be fixed with a more carefully crafted prompt to form the KG. As mentioned in my earlier blog, I had generated the prompt from chatGPT and used the same one with minor changes. For real-world use case one should spend more time prompt formation. Also, we could try a bigger model for enterprise use cases as resource constraint won't be an issue there.

In conclusion, this approach seems promising and could be mixed with RAG to improve the quality of responses. May be in my next blog, I will try to improve

Let me know if you have any interesting ideas, suggestions in the comments.
Don't forget to give claps if you found this helpful.

[#artificial-intelligence](#) [#machine-learning](#) [#data-science](#) [#technology](#) [#knowledge-graph](#)